UNIVERSITY OF CENTRAL ARKANSAS
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

# Assignment #4: Software Implementation with Generative AI (Group Assignment)

**Proposed System:** Patient Record System (PRS)

**Purpose:** In Assignment #2, your group developed the requirements specification for the Patient Record System (PRS). In Assignment #3, you designed its software architecture using the 4+1 View Model and UML diagrams.

In this assignment, you will implement your PRS Software **using Object-Oriented Programming (OOP) with assistance from Generative AI tools such as ChatGPT or Claude**.

You will generate an initial implementation, examine the code, identify problems, correct and refactor it, and produce an improved version. You must demonstrate that you understand the software you developed, including its classes, methods, relationships, design decisions, and limitations.

Use the **same group, requirements, and architecture** from Assignments #2 and #3.

**Academic Integrity: Independent Code Analysis**

Generative AI can be used to generate, debug, correct, optimize, and refactor source code. However, all written code analysis, identification of weaknesses, technical explanations, comparisons, and reflections must be completed independently by your group.

**AI-generated written analysis or explanations are prohibited and WILL result in a grade of ZERO (0) for Assignment #4 following academic integrity review.**

You should have the ability to explain the submitted code and justify your implementation decisions.

## Part 1: Initial Software Implementation Using GenAI

### 1.1 Prepare the Implementation

Provide GenAI with your finalized requirements from Assignment #2 and the architectural diagrams from Assignment #3. Use **Python and OOP** to develop the PRS. Your implementation should follow the approved requirements and architecture, including the following functionality:

1. **Patient Management:** Create patient records, generate unique and immutable Patient IDs, retrieve records, and update permitted patient information.
2. **Visit Note Management:** Add dated visit notes only to existing patients, maintain the correct patient-note relationships, and retrieve visit history.
3. **Authentication and Authorization:** Identify staff users and enforce appropriate permissions for protected operations.
4. **AI-Assisted Categorization:** Suggest one of four categories (Respiratory, Digestive, Musculoskeletal, General/Other). Staff must accept or reject suggestions, and unavailable or uncertain AI must not prevent saving visit notes.

5. **Data Persistence:** Store patient records and visit notes so that data remains available after the application restarts.

Your implementation may use an approved AI API, a locally running model, or a suitable pretrained model for categorization. A hardcoded keyword-based classifier may be used only as a clearly identified temporary fallback or test substitute, not presented as an actual AI model. **Use synthetic patient information only.**

### 1.2 Generate Version 1

Use GenAI to generate the initial working application based on your requirements and architecture. Your Version 1 must include:

- Object-oriented classes and methods with defined responsibilities.
- Appropriate relationships between classes and components.
- A working interface (web, desktop, or command-line) to demonstrate the required operations.
- Database or equivalent persistent storage.
- Instructions for installing and running the application.

**Deliverable:** Preserve the original AI-generated code as **Version 1**, before making corrections or improvements.

## Part 2: Analyze the Generated Code

Do not immediately ask GenAI to improve Version 1. Your group must first examine the code and identify its strengths, weaknesses, and errors.

### 2.1 Analyze the OOP Implementation

Identify the major classes in your implementation and complete the following table:

| Class | Main attributes | Important methods | Responsibility | OOP principles applied |
|---|---|---|---|---|
| **Patient** | ... | ... | ... | ... |
| **VisitNote** | ... | ... | ... | ... |
| **Additional classes** | ... | ... | ... | ... |

Explain how your implementation uses the four fundamental OOP principles:

- **Encapsulation:** How is access to internal object state controlled?
- **Abstraction:** Which interfaces or methods hide implementation details?
- **Inheritance:** Where is inheritance used, if appropriate, and why?
- **Polymorphism:** Where can different implementations be used through a common interface?

Do not introduce unnecessary inheritance or polymorphism merely to demonstrate these concepts. If your design does not require one, explain why.

### 2.2 Mandatory Analysis: Immutable Patient ID

The PRS requires each Patient ID to be unique and permanent.

**Explain specifically:**

1. Which class and attribute represent the Patient ID?
2. Which OOP mechanism or Python technique did you use to prevent modification after object creation?

3. Identify the exact method, property, or code responsible for enforcing immutability.
4. Can the Patient ID still be modified through direct attribute access, another method, or the database?
5. How does the system prevent two patients from receiving the same ID?
6. Demonstrate what happens when someone attempts to change an existing Patient ID.

**Important:** Encapsulation, immutability, and uniqueness are related but different concepts. Your explanation must distinguish them. A naming convention such as _patient_id alone does not guarantee immutability.

### 2.3 Analyze Code Quality and Implementation Decisions

Identify **at least five code-specific findings**, including implementation weaknesses, design limitations, or justified opportunities for improvement. Each finding must include the relevant class or method, your analysis, and a proposed action.

1. **OOP Design:** Incorrect class responsibilities, weak encapsulation, unnecessary inheritance, poor abstraction, or tightly coupled classes.
2. **Code Structure and Maintainability:** Duplicated code, overly long methods, unnecessary complexity, or poorly organized modules.
3. **Programming Standards:** Inconsistent naming, formatting, documentation, exception handling, or other violations of Python programming conventions.
4. **Efficiency and Optimization:** Unnecessary database operations, repeated computations, inefficient data structures, or avoidable processing.
5. **Requirements and Architecture Alignment:** Missing behavior, inappropriate dependencies, or implementation decisions inconsistent with Assignments #2 and #3.

For each finding, identify the **class/method and relevant code**, explain why the implementation is problematic, and propose a specific improvement. Your analysis must be based on your own examination of the generated code, not simply copied from GenAI.

## Part 3: Code Review and Implementation Analysis

Examine the generated source code and explain how the major requirements and architectural components were implemented.

### 3.1 Requirements and Architecture to Code

Select at least **five important requirements** from Assignment #2 and identify where they are implemented.

| Requirement ID | Class/Module | Method | How it is implemented |
|---|---|---|---|
| **FR-XX** | PatientService | register_patient() | ... |
| **...** | ... | ... | ... |

Explain any differences between your Assignment #3 architecture and the generated implementation. If GenAI introduced additional classes, dependencies, or design changes, determine whether they are necessary.

### 3.2 Detailed Code Review

Select two important classes or methods from Version 1, preferably ones identified in Part 2.3. For each, explain:

- Its responsibility and relationship to other components.
- Whether its OOP design and implementation are appropriate.
- The specific code that should be retained, corrected, or improved, and why.

Reference the relevant source code. You may use findings from Part 2.3 rather than identifying additional problems.

## Part 4: Correct, Optimize, and Refactor with GenAI

### 4.1 Generate Version 2

After completing your independent code analysis, provide GenAI with:

- Version 1 source code.
- The implementation weaknesses identified in Parts 2 and 3.
- Relevant requirements and architectural constraints.
- Specific instructions for improving the code.

Use GenAI to produce **Version 2**, addressing the identified issues through appropriate corrections, refactoring, and optimization. Focus on:

1. **OOP Improvement:** Improve encapsulation, abstraction, class responsibilities, and dependencies.
2. **Refactoring:** Remove duplicated code, simplify overly complex methods, and improve code organization without changing intended behavior.
3. **Optimization:** Eliminate unnecessary operations or improve inefficient code where justified.
4. **Programming Standards:** Improve naming, formatting, error handling, documentation, and consistency.
5. **Architecture Alignment:** Ensure the revised classes and modules remain consistent with the approved system design.

Do not introduce unnecessary complexity or new features during refactoring.

Save the revised implementation as **Version 2**.

### 4.2 Compare Version 1 and Version 2

Document the meaningful corrections, refactoring, and optimization performed in Version 2, addressing your findings from Part 2.3. Explain any recommendations you decided not to implement.

| Class/ Method | Problem in Version 1 | Improvement in Version 2 | Type of change | Why it is better |
|---|---|---|---|---|
| **Patient** | Patient ID is publicly writable | Restrict ID modification and enforce storage-level immutability | Correction/OOP improvement | Protects permanent patient identity |
| **...** | ... | ... | ... | ... |

Classify changes as **defect correction, refactoring, optimization, programming-standard improvement, or architectural improvement**, as appropriate.

Explain the actual benefit of each change. Do not claim improved performance without supporting evidence.

### 4.3 Final Implementation Review

Run Version 2 and demonstrate that the major PRS operations are available, including patient registration, record retrieval, visit-note management, and AI categorization or fallback. Briefly explain:

- Whether the final application executes successfully.
- Whether the identified implementation problems have been addressed.
- Whether any important limitations remain.

Formal test-case design, automated testing, coverage analysis, and test-result reporting will be addressed in **Assignment #5: Software Testing**.

## Part 5: Demonstrate Your Understanding

Provide concise, code-specific explanations addressing the following:

1. **Architecture to Implementation:** How were your requirements and UML designs translated into classes, methods, interfaces, and modules? What changed during implementation, and why?
2. **OOP Decisions:** How did you apply encapsulation and abstraction? Where were inheritance and polymorphism appropriate, or why were they unnecessary?
3. **Patient ID Immutability:** Explain precisely how your final implementation prevents changes to an existing Patient ID and prevents duplicate IDs. Identify the responsible code and database constraints.
4. **Critical Code Analysis:** What were the most important problems in Version 1? How did you identify them independently?
5. **Refactoring and Optimization:** Select two significant improvements in Version 2. Explain the original code, the revised approach, and the technical reason for each change.
6. **Code Understanding:** Explain the execution of one important operation, from its interface through the relevant classes and methods to data persistence.
7. **GenAI Contribution:** What code did GenAI produce? Which improvements did your group identify or make, and what engineering decisions did you make independently?

Use actual class names, method names, code excerpts, and requirement IDs. **Your analysis and explanations must be written by your group, not generated by AI.**

## Part 6: Submission Requirements

Submit one PDF report per group containing:

1. Group members and project information.
2. Version 1 implementation overview and major classes.
3. OOP analysis, including Patient ID immutability and uniqueness.
4. Independent analysis of at least five implementation weaknesses.
5. Requirements-to-code mapping and detailed review of two important classes/methods.
6. Version 2 corrections, refactoring, and optimization.
7. Version 1 vs. Version 2 comparison.
8. Technical understanding and GenAI contribution.
9. Principal GenAI prompts used for code generation, correction, and refactoring.

Include your **public GitHub repository link** in the report. The repository must contain both versions of the source code and instructions for running the application.